Supervised Fine-Tuning (SFT)

Teaching LLMs to Formulate Text & Follow Instructions

Yakov (Dima)

September 2026

Two Purposes of SFT

Alignment SFT

Base completion model → general instruction-following assistant.

Teaches the model to act on instructions, not just continue text. Foundational, one-time.

Domain / Application SFT

Instruction-tuned model → specialized behavior for one product.

Already follows instructions — this adds *your* schema, vocabulary, and edge cases.

Same mechanism (masked next-token loss on instruction data) — just a different starting checkpoint and goal.

Alignment SFT: Pre-training vs. SFT (The Mindset Shift)

Base LLM — Text Completer

Prompt: Write a python function for quicksort.

Completion: Write a python function for mergesort.
Write a python function for binary search.
Write a python function for...

Chaos pattern: keeps completing the list instead of answering it.

SFT LLM — Helpful Assistant

Prompt: Write a python function for quicksort.

Completion:

```
def quicksort(arr):  ...  return arr
```

Alignment pattern: playing the persona of an assistant.

Domain SFT: An Autonomous-Driving Copilot

General Instruct Model (Alignment SFT only)

Prompt: Pedestrian at crosswalk distracted; truck in bike lane 20m ahead.

Completion: Moderate risk — distracted pedestrian near the crosswalk, truck blocking the bike lane ahead. Recommend slowing down.

Limitation: correct but free-form prose — not parseable by a planner.

Domain-SFT'd Driving Copilot

Prompt: (same)

Completion:

```
{"actors": [{"type":
"VRU_pedestrian", "risk":
"high"}, {"type":
"static_obstacle", "risk":
"medium"}], "action": "reduce
speed; widen clearance"}
```

Result: fine-tuned on the AV stack's own logs.

Domain SFT only had to teach *this* team's risk taxonomy and output format.

Training: Same Loss, Adapted Rules

```
<|user|> What is 2+2?  <|assistant|> The answer is 4.
```

$$\mathcal{L} = -\sum_{t=i}^{N} \log P(x_t \mid x_{<t})$$

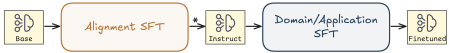
No new objective: same cross-entropy next-token prediction as pre-training.

Masking: loss computed only on assistant tokens.

Chat template tokens mark where turns begin/end, for masking and stopping.

Learning rate: dropped 1–2 orders of magnitude vs. pre-training.

Supervised Fine-Tuning (SFT)



Training: Same CE Loss, Adapted Rules

The LLM Alignment Pipeline



Pre-training: web-scale raw knowledge and language structure acquisition.

SFT: teaching interaction formats, tasks, and system behaviors.

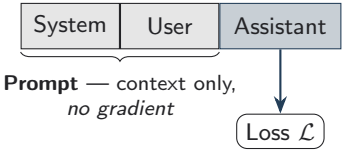
RLHF/DPO: micro-tuning behavior to meet safety standards and human feedback.

This 3-stage pipeline was introduced by Ouyang et al., *Training language models to follow instructions with human feedback* (InstructGPT, 2022).

The Anatomy of an Instruction Example

ChatML / OpenAI JSON template structure

```
[
  {"role": "system", "content": "You are a helpful coding assistant."},
  {"role": "user", "content": "How do I reverse a list in Python?"},
  {"role": "assistant", "content": "You can reverse a list using
     ‘list.reverse()‘ or slicing ‘[::-1]‘."}
]
```



The Gold Target *— loss is calculated ONLY on the assistant response.*

System: defines boundaries/tone.

User: input instructions — part of the context, not the target.

Assistant: the training target — this is what the loss actually optimizes.

Instruction Tuning Works: The FLAN Result

FLAN — Finetuned Language Models Are Zero-Shot Learners (Wei et al., 2022).

Instruction tuning generalizes across tasks — not just new examples, but entirely new task types.



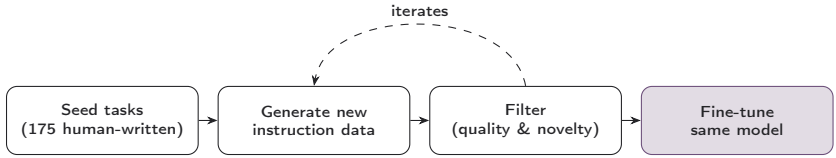
Result: Holding out entire task clusters (e.g., all NLI), zero-shot FLAN beats GPT-3 on **20 of 25** held-out types — even topping few-shot on several.

Ablations confirm real generalization: gains grow with task diversity and require natural-language phrasing.

Bootstrapping Instructions from the Model Itself: Self-Instruct

Self-Instruct (Wang et al., 2023).

Have the model generate its own training data, then filter.



Result: +33.1% ROUGE-L over GPT-3 on Super-NaturalInstructions — within 5% of InstructGPT on novel tasks, minimal annotation.

Direct ancestor of the Llama 3 rejection-sampling pipeline (two slides ahead) — less refined.

Where the Data Actually Comes From

Both reports use mostly model-generated data — humans verify or select, not write from scratch.

DeepSeek-V3 — Model Generates, Humans Verify

Reasoning (math/code/logic): generated by **DeepSeek-R1**, an internal AI reasoning model, then rejection-sampled.

Non-reasoning (writing, role-play, QA): generated by **DeepSeek-V2.5**; humans *verify* correctness, don't write it.

Llama 3 — Humans Write Prompts, Models Draft Responses

Prompts: collected via human annotation — real-world-style use cases.

Responses: mostly **rejection-sampled** (K=10–30, RM picks best) — not human-written.

Plus targeted **synthetic data** (code, long-context, tools) and a small hand-curated slice.

Both pipelines diverge from FLAN: humans supply prompts, models draft responses, then filter via sampling or verification.

DeepSeek-V3: Quantity vs. Quality, by the Numbers

A case study from one published model report — DeepSeek-V3.

Stage	Data	Compute
Pre-training	14.8T tokens	2,664K H800-hrs → $5.33M
Context extension	YaRN, 2 phases × 1000 steps: 4K→32K→128K	119K H800-hrs → $0.24M
Post-training (SFT+RL)	1.5M instruction instances	5K H800-hrs → $0.01M

Context extension: no new data, just longer sequences — the model re-learns to attend over a longer window before it's taught to follow instructions. *Caveat: the post-training figure bundles SFT and RL; no SFT-only cost is published.*

DeepSeek-V3 at a glance — MoE, 671B total / 37B active params · 128K-token context · 128K BPE vocabulary

What "Curated" Actually Means

A case study from one published model report — Llama 3.

Pre-training Data

Scraped from the web — bulk, automatic, corpus-level

Dedup (URL / doc via MinHash / line)
↓
Heuristic filters (n-grams, dirty words, KL)
↓
fastText / RoBERTa classifiers

SFT Data

Written/generated on purpose — per-example, model-judged

Rejection sampling (K=10–30)
↓
Clean & topic-tag
↓
Quality × difficulty score
↓
Semantic dedup

} ×6 rounds

Llama 3's own summary: *"more careful pre-processing and curation pipelines for pre-training data . . . more rigorous quality assurance and filtering approaches for post-training data."* §3.1 / §4.2.3.

Quality Over Quantity: The LIMA Effect

LIMA (Zhou et al., 2023): 1,000 curated examples outperform Alpaca's 52K and stay competitive with RLHF'd models like DaVinci003 and Bard.

Equal-or-Preferred Rate — % of matchups where LIMA tied or won in blind comparisons.

vs. baseline	DaVinci003 (RLHF)	Bard	Claude	GPT-4
LIMA equal-or-preferred	**65%**	**58%**	46%	43%

65B LLaMA, 1,000 examples, no RLHF.

After SFT: Why RL Still Helps

Alignment SFT

Instruction-following, persona

Broad behavioral patterns
Overfits to training details

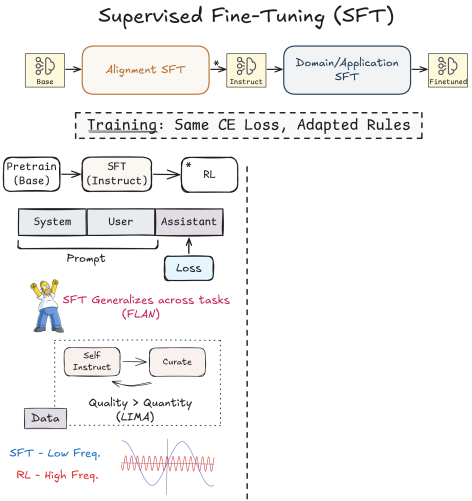
RL / RLHF

Preferences, refusals, style

Fine-grained calibration
Adapts to variation

SFT stabilizes format; RL calibrates boundaries.

SFT captures **low-frequency patterns** (instruction-following), RL calibrates **high-frequency variation** (preferences, refusals, style) — complementary. TailSFT (Malladi et al., 2026) filters well-fit sequences, improving pass@K for RL.



RLHF

nces, refusals, style

ined calibration

o variation

brates **high-frequency variation**

l., 2026) filters well-fit sequences,

What Alignment SFT + RL Don't Give You

Alignment SFT (low-frequency)
Instruction-following, persona

RL / RLHF (high-frequency)
Preferences, refusals, style

Still missing:

Your output schema (JSON, XML, domain-specific DSL)

Your vocabulary (legal terms, medical codes, internal jargon)

Your edge cases (product-specific failure modes)

Optimized accuracy on *your* task distribution

The driving copilot (slide 4): can answer, but not in *your* schema.

Domain Adaptation Still Helps

Key finding *(Gururangan et al., 2020):*
Continued domain- or task-specific training measurably improves performance.

Adaptation Type	F1 Gain	Method
Domain Adaptive Pretraining (DAPT)	+1–8	Domain corpus
Task Adaptive Pretraining (TAPT)	+1–3	Task corpus
DAPT + TAPT	Best	Both stacked

Domain SFT delivers both:

Format compliance (schema, vocabulary)

Task accuracy (measurable gains)

Prompt Engineering vs. Domain SFT

The fundamental alternative: Use the context window instead of changing weights.

Prompt engineering is a family of techniques:

Zero-shot instructions: "Translate to French: Hello" — just describe the task

Few-shot examples (ICL): "cat→chat, dog→chien, bird→?" — demonstrate the pattern

Min et al., 2022: labels barely matter; format & distribution do

Chain-of-Thought: "Let's think step by step..." — elicit reasoning

Role-playing: "You are an expert Python tutor..." — persona framing

Format specification: "Output JSON: {risk, action}" — structure constraints

Rules: "Answer using ONLY the documents below" — behavior boundaries

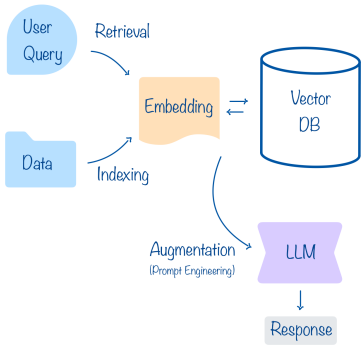
All share the same tradeoff: $0 upfront, instant updates, but you pay per request and the model can drift. SFT bakes behavior into weights once.

Retrieval-Augmented Generation (RAG)

Idea: Dynamic knowledge injection — retrieve relevant documents per query, inject into context.

What makes RAG different *(Lewis et al., 2020)*:

- Knowledge lives in the index, not the weights — **instant updates**
- Can cite sources — **provenance** and reduced hallucination



Mechanistically, RAG is prompt engineering with retrieval — dynamic ICL for knowledge grounding.

Explore more: RAG Variants | Vector Databases & Similarity Search

Summary: When to Use What

Criterion	Prompt Eng.	RAG	Domain SFT
Modifies weights?	No	No	**Yes**
Upfront cost	None	Index build	**High (data + training)**
Per-request cost	High (examples)	Medium (retrieval)	**Low (short prompts)**
Update latency	Instant	Instant	Retrain
Best for	Task steering	**Knowledge**	**Behavior**
Reliability	Can drift	Depends on retrieval	**Durable (in weights)**

Prompt Engineering includes: zero-shot instructions, few-shot ICL, chain-of-thought, role-playing, format specs, rules

Rule of thumb:

Quick iteration / low volume → Prompt Engineering

Knowledge (facts, documents that change) → RAG

Behavior (format, style, reasoning) → SFT

Production → SFT for behavior + RAG for knowledge

Why Not Full Fine-Tuning?

Full fine-tuning = update every parameter in the model.

Problems at scale:

Memory: GPT-3 175B needs ~700GB VRAM just for gradients + optimizer states

Catastrophic forgetting: model loses general capabilities when overtrained on narrow data

Storage: need a separate 175B-parameter checkpoint per task/customer

Cost: full training runs are expensive to repeat

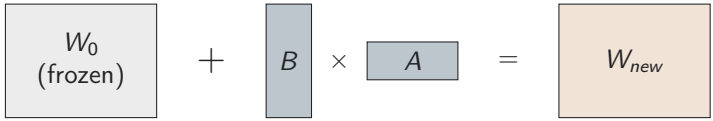
Solution: Parameter-Efficient Fine-Tuning (PEFT)

Freeze most of the model; train only a small set of new or selected parameters.

PEFT is still SFT—same loss function, same training data. Only the *scope* of what gets updated changes.

LoRA: Low-Rank Adaptation

Core idea *(Hu et al., 2021):* Weight updates are often low-rank—decompose ΔW into two small matrices.



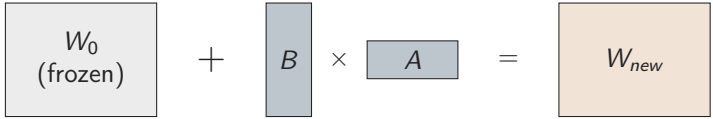
$$W_{new} = W_0 + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r \times d_{in}}, \; B \in \mathbb{R}^{d_{out} \times r}$$

Key properties:

- **Trainable params:** $\sim$0.01% of full model (rank r typically 4–64)
- **Memory:** 3$\times$ less GPU memory than full fine-tuning
- **Inference:** merge BA into $W_0 \rightarrow$ **zero added latency**
- **Performance:** matches or exceeds full fine-tuning on many tasks

LoRA: Low-Rank Adaptation

Core idea *(Hu et al., 2021):* Weight updates are often low-rank—decompose ΔW into two small matrices.



$$W_{new} = W_0 + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r \times d_{in}}, \; B \in \mathbb{R}^{d_{out} \times r}$$

Which weights to adapt?

Targeted: LoRA applied per-weight-matrix, each gets its own (A, B) pair

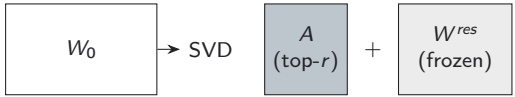
Attention first: W_q, W_k, W_v, W_o (standard targets)

MLP optional: larger, holds world knowledge; costs more but helps when task needs new knowledge

LoRA Variants: PiSSA (SVD-Based Initialization)

Problem with vanilla LoRA: A initialized with Gaussian noise, B with zeros $\rightarrow$ slow convergence.

PiSSA *(Meng et al., 2025):* Initialize adapters with the **principal singular components** of W_0 itself.



Results:

Gemma-7B on GSM8K: PiSSA **77.7%** vs. LoRA 74.5% (+3.2%)

LLaMA-3-70B with 4-bit quant: QPiSSA **86.1%** vs. QLoRA 81.7% (+4.4%)

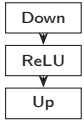
Initialization cost: seconds (fast SVD), negligible vs. training time

Same architecture as LoRA—drop-in replacement. The only change is *how* A and B are initialized.

Other PEFT Families

Adapters
(Houlsby et al., 2019)

Insert small bottleneck layers *inside* transformer blocks.



∼3% params per task.
Adds inference latency.

Prefix-Tuning
(Li & Liang, 2021)

Prepend learnable "virtual tokens" to keys/values in every layer.



∼0.1% params.
No architecture change.

Prompt Tuning
(Lester et al., 2021)

Learnable soft prompt at input layer only (not every layer).

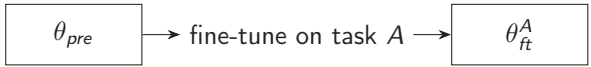
Simpler than prefix-tuning. Scales with model size: "closes the gap" at 10B+ params.

∼0.01% params.
Minimal overhead.

All methods freeze the base model and train only a small number of new parameters. Choice depends on your latency/memory constraints and how much expressiveness you need.

Task Arithmetic: Combining Adapters

Core idea *(Ilharco et al., 2023):* A **task vector** = fine-tuned weights − pretrained weights.



$$\tau_A = \theta_{ft}^{A} - \theta_{pre}$$

Operations on task vectors:

Negation: $\theta_{pre} - \tau_A \rightarrow$ *reduces* performance on task A

Addition: $\theta_{pre} + \tau_A + \tau_B \rightarrow$ improves on *both* A and B

Analogy: "A is to B as C is to D" $\rightarrow$ combine τ_A, τ_B, τ_C to improve on D

No retraining needed—just arithmetic on saved weight deltas.

Works with LoRA too: each adapter is a task vector you can combine post-hoc.

Advanced Weight Merging Techniques

Problem: Naive addition of task vectors causes interference—parameter conflicts between tasks.

Method	How It Works	Key Benefit
SLERP	Spherical interpolation—preserves weight norm along a curved path	No magnitude squashing
TIES-Merging	(1) Trim small deltas (2) Elect dominant sign (3) Merge agreeing params	Resolves sign conflicts
DARE	Drop 90% of deltas randomly, rescale survivors by $1/(1-p)$	Extreme sparsity $\rightarrow$ less interference

Applications: Unlearning and Safety

Task vectors enable **editing models without retraining**.

Targeted Forgetting

Goal: Remove toxic, copyrighted, or private data from a trained model.

Method:

- ▷ Fine-tune model on unwanted data $\rightarrow$ θ_{bad}
- ▷ Compute $\tau_{bad} = \theta_{bad} - \theta_{base}$
- ▷ Apply: $\theta_{new} = \theta_{base} - \lambda\tau_{bad}$

Pushes weights *away* from the learned representations.

Alignment without RLHF

Goal: Add safety guardrails to an unaligned model without access to RLHF data.

Method:

- ▷ Isolate "alignment" task vector τ_{safe} from safe model
- ▷ Scale into unaligned base: $\theta_{new} = \theta_{base} + \lambda\tau_{safe}$

Transfers safety properties across models.

Both leverage the linear structure of task vectors—behavior encoded as directions in weight space can be added, subtracted, or scaled.

Summary: PEFT Method Comparison

Method	Params	Memory	Inference	Where	Composable?
Full FT	100%	Very high	None	All	No
LoRA	0.01–0.1%	3× lower	**None** (merge)	Attention	**Yes**
PiSSA	0.01–0.1%	3× lower	**None**	Attention	Yes
Adapters	1–5%	Medium	Added	Each layer	Yes
Prefix-Tuning	0.1%	Low	Minimal	K/V cache	Yes
Prompt Tuning	0.01%	Very low	None	Input only	Yes

Practical defaults:

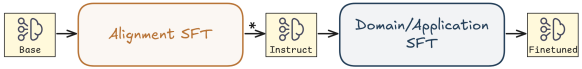
Start with LoRA (r=8–32): best balance of efficiency and performance

Try PiSSA if LoRA converges slowly: same cost, faster training

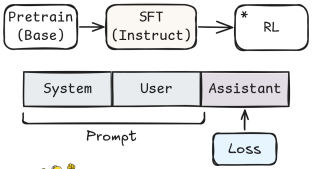
Use Adapters if you need maximum expressiveness and can tolerate latency

Task Arithmetic to combine multiple LoRA/adapters without retraining

Supervised Fine-Tuning (SFT)

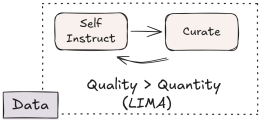


Training: Same CE Loss, Adapted Rules



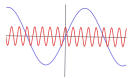


SFT Generalizes across tasks (FLAN)



SFT - Low Freq.

RL - High Freq.



Why we need this finetuning step?

Customization

Output Schema	Vocabulary
Edge Cases	Accuracy

Alternative: Prompt Eng.

Zero-shot	ICL	CoT	RAG
Role-playing	Format	Rules	

PEFT

LoRA Adapters Prefix-Tuning Prompt Tuning



- Random init
- SVD

Task Arithmetic (Weight Merging): $\theta + \tau$

RAG Variants: Active, Hierarchical, Graph-Based

The basic RAG pattern has many extensions:

Variant	Key Idea	Best For
Self-RAG (Asai et al., 2023)	Model decides *when* to retrieve; critiques own outputs	Reducing unnecessary retrieval
FLARE (Jiang et al., 2023)	Retrieves *during* generation when confidence drops	Long-form generation
RAPTOR (Sarthi et al., 2024)	Hierarchical tree of summaries, not flat chunks	Multi-hop reasoning
Graph RAG (Edge et al., 2025)	Entity graph + community summaries	Complex entity relationships

All variants share the core idea: inject external knowledge at inference time rather than baking it into weights. Choice depends on your retrieval patterns and reasoning complexity.

← Back to RAG

Vector Databases & Similarity Search

Approximate Nearest Neighbor (ANN) search enables efficient retrieval at scale.

Common similarity measures:

- **Euclidean Distance** — geometric distance in vector space
- **Cosine Similarity** — angle between vectors (ignores magnitude)
- **Dot Product** — combines magnitude and direction

Indexing methods (sacrifice perfect accuracy for speed):

Method	Core Idea	How it Works	Best For
HNSW	Graph	Navigates multi-layered graph to find neighbors	Performance & accuracy-critical (common default)
IVF	Clustering	Searches only within relevant vector clusters	Very large datasets, memory-constrained
LSH	Hashing	Hashes similar vectors into same bucket	Massive scale, speed over perfect accuracy

Trade-off: exact search is $O(n)$; ANN methods achieve sub-linear query time with controllable accuracy loss.

← Back to RAG